Data Structures

BS (CS) _Fall_2025

Lab_04 Task

Learning Objectives:

1. Array Based List
2. Exception handling

Task 1: Array Based List using Templates

You are required to design and implement a generic List class using templates in C++. The list should be array-based and provide basic as well as extended list operations.

Class Requirements

Your List<T> class should have the following data members:

- **T* arr :** a pointer to the dynamic array (to store list elements).
- **int capacity:** stores the maximum size of the list.
- **int counter:** keeps track of the current number of elements.

Memory for the array should be allocated in the constructor.

Operations to Implement

Insertion Functions

1. **insert(T item)**
   Inserts an item at the end of the list (if not full). Increments the counter.
2. **insertAt(T item, int index)**
   Inserts an item at a given index (without overwriting). All elements after that index must be shifted to create space.
3. **insertAfter(T itemToInsert, T existingItem)**
   Finds existingItem in the list and inserts itemToInsert after it.
   (Reuse your own insertAt instead of duplicating logic).
4. **insertBefore(T itemToInsert, T existingItem)**
   Finds existingItem in the list and inserts itemToInsert before it.
   (Reuse your own insertAt here as well).

Removal Functions

5. **remove(T item)**
   Removes the given item if it exists in the list. Shifts elements left and decrements counter.
6. **removeBefore(T item)**
   Finds item and removes the element just before it.
   Example: {1, 2, 3, 4}, removeBefore(3) :- {1, 3, 4}
7. **removeAfter(T item)**
   Finds item and removes the element just after it.
   Example: {1, 2, 3, 4, 5}, removeAfter(3) :- {1, 2, 3, 5}

Utility Functions

8. **isEmpty()**
   Returns true if the list has no elements.
9. **isFull()**
   Returns true if the list has reached its maximum capacity.
10. **search(T item)**
    Returns the index of the item if found, otherwise -1.
11. **print()**
    Prints all elements of the list.
12. **reverse()**
    Reverses the list in-place.

Gtest:

```
// ---------- Insert Tests ----------
TEST(ListTest, InsertAtEnd) {
    List<int> lst(5);
    lst.insert(10);
    lst.insert(20);
    lst.insert(30);
    EXPECT_EQ(lst.search(10), 0);
    EXPECT_EQ(lst.search(20), 1);
    EXPECT_EQ(lst.search(30), 2);
    EXPECT_EQ(lst.isFull(), false);
}

TEST(ListTest, InsertAtSpecificIndex) {
    List<int> lst(5);
    lst.insert(10);
    lst.insert(20);
    lst.insertAt(15, 1); // insert at index 1
    EXPECT_EQ(lst.search(15), 1);
    EXPECT_EQ(lst.search(20), 2); // shifted
}

TEST(ListTest, InsertAfterElement) {
    List<int> lst(5);
    lst.insert(10);
    lst.insert(20);
    lst.insert(30);
    lst.insertAfter(25, 20);
    EXPECT_EQ(lst.search(25), 2);
}
```

```
TEST(ListTest, InsertBeforeElement) {
    List<int> lst(5);
    lst.insert(10);
    lst.insert(20);
    lst.insert(30);
    lst.insertBefore(15, 20);
    EXPECT_EQ(lst.search(15), 1);
}

// ---------- Remove Tests ----------
TEST(ListTest, RemoveElement) {
    List<int> lst(5);
    lst.insert(10);
    lst.insert(20);
    lst.insert(30);
    lst.remove(20);
    EXPECT_EQ(lst.search(20), -1);
    EXPECT_EQ(lst.search(30), 1); // shifted left
}

TEST(ListTest, RemoveBefore) {
    List<int> lst(5);
    lst.insert(1);
    lst.insert(2);
    lst.insert(3);
    lst.insert(4);
    lst.removeBefore(3);
    EXPECT_EQ(lst.search(2), -1);
    EXPECT_EQ(lst.search(3), 1); // shifted left
}

TEST(ListTest, RemoveAfter) {
    List<int> lst(5);
    lst.insert(1);
    lst.insert(2);
    lst.insert(3);
    lst.insert(4);
    lst.insert(5);
    lst.removeAfter(3);
    EXPECT_EQ(lst.search(4), -1);
    EXPECT_EQ(lst.search(5), 3);
}

// ---------- Utility Tests ----------
```

```
TEST(ListTest, IsEmptyAndIsFull) {
    List<int> lst(3);
    EXPECT_TRUE(lst.isEmpty());
    lst.insert(1);
    lst.insert(2);
    lst.insert(3);
    EXPECT_TRUE(lst.isFull());
}

TEST(ListTest, SearchNotFound) {
    List<int> lst(5);
    lst.insert(100);
    EXPECT_EQ(lst.search(200), -1);
}

TEST(ListTest, ReverseList) {
    List<int> lst(5);
    lst.insert(1);
    lst.insert(2);
    lst.insert(3);
    lst.reverse();
    EXPECT_EQ(lst.search(3), 0);
    EXPECT_EQ(lst.search(1), 2);
}
```

Task 2: Extended Array-Based List

In this task, you will extend your List<T> class from Task 1 by implementing more advanced functionalities. The goal is to make your class robust, generic, and exception-safe.

General Requirements

- **Templates:** Use template <typename T> so the list works with all data types.
- **Exception Handling:** Every function must include exception handling using C++ standard exceptions:
  - std::out_of_range :- invalid index or invalid range.
  - std::overflow_error :- inserting into a full list.
  - std::underflow_error :- removing/finding from an empty list.
  - std::invalid_argument :- invalid arguments, e.g., replacing an element that doesn't exist.

New Operations & Exception Requirements

1. **rotateLeft(int k)**
   - **Description**: Rotates the list k steps to the left.
   - **Exceptions**:
     - Throw std::underflow_error if the list is empty.
     - Throw std::out_of_range if k < 0 or k > counter.
   - **Example**:

```
List<int> lst(5);
lst.insert(10);
lst.insert(20);
lst.insert(30);
lst.insert(40);

lst.rotateLeft(2);
// Result: [30, 40, 10, 20]
```

2. **sort(bool ascending = true)**
   - **Description:** Sorts the list in ascending or descending order
   - **Exceptions:**
     - Throw std::underflow_error if the list is empty.
   - **Example:**

```
List<int> lst(5);
lst.insert(40);
lst.insert(10);
lst.insert(30);

lst.sort(true);
// Result: [10, 30, 40]

lst.sort(false);
// Result: [40, 30, 10]
```

3. **removeDuplicates()**
   - **Description**: Removes duplicate elements while keeping only the first occurrence.
   - **Exceptions**:
     - Throw std::underflow_error if the list is empty.

- **Example**:

```
List<int> lst(6);
lst.insert(10);
lst.insert(20);
lst.insert(10);
lst.insert(30);
lst.insert(20);

lst.removeDuplicates();
// Result: [10, 20, 30]
```

4. **subList(int start, int end)**
   - **Description:** Returns a new List<T> containing elements between indices start and end (inclusive)
   - **Exceptions:**
     - Throw std::underflow_error if the list is empty.
     - Throw std::out_of_range if start or end are invalid indices.
     - Throw std::out_of_range if start > end.
   - **Example:**

```
List<int> lst(6);
lst.insert(5);
lst.insert(10);
lst.insert(15);
lst.insert(20);

auto sub = lst.subList(1, 2);
// Result: [10, 15]
```

5. **findMax() / findMin()**
   - **Description**: Finds and returns the maximum or minimum element in the list.
   - **Exceptions**:
     - Throw std::underflow_error if the list is empty.
   - **Example**:

```
List<int> lst(4);
lst.insert(25);
lst.insert(10);
```

```
lst.insert(35);

cout << lst.findMax();  // Output: 35
cout << lst.findMin();  // Output: 10
```

6. **replace(T oldItem, T newItem)**
    - **Description**: Replaces all occurrences of oldItem with newItem.
    - **Exceptions**:
        - Throw std::invalid_argument if oldItem is not found in the list.
    - **Example**:

```
List<int> lst(5);
lst.insert(5);
lst.insert(10);
lst.insert(5);

lst.replace(5, 50);
// Result: [50, 10, 50]
```

Gtest for Task 2:

```
/
/ ---------------- INT TESTS ----------------

// Rotate Left
TEST(ExtendedListTest_Int, RotateLeftValid) {
    List<int> lst(5);
    lst.insert(10);
    lst.insert(20);
    lst.insert(30);
    lst.insert(40);
    lst.rotateLeft(2);
    EXPECT_EQ(lst.search(30), 0);
    EXPECT_EQ(lst.search(10), 2);
}

TEST(ExtendedListTest_Int, RotateLeftEmpty) {
    List<int> lst(5);
    EXPECT_THROW(lst.rotateLeft(1), std::underflow_error);
}
```

```
TEST(ExtendedListTest_Int, RotateLeftInvalidK) {
    List<int> lst(5);
    lst.insert(10);
    lst.insert(20);
    EXPECT_THROW(lst.rotateLeft(-1), std::out_of_range);
    EXPECT_THROW(lst.rotateLeft(10), std::out_of_range);
}

// Sort
TEST(ExtendedListTest_Int, SortAscendingDescending) {
    List<int> lst(5);
    lst.insert(40);
    lst.insert(10);
    lst.insert(30);
    lst.sort(true);
    EXPECT_EQ(lst.search(10), 0);
    EXPECT_EQ(lst.search(40), 2);

    lst.sort(false);
    EXPECT_EQ(lst.search(40), 0);
    EXPECT_EQ(lst.search(10), 2);
}

TEST(ExtendedListTest_Int, SortEmpty) {
    List<int> lst(5);
    EXPECT_THROW(lst.sort(), std::underflow_error);
}

// Remove Duplicates
TEST(ExtendedListTest_Int, RemoveDuplicates) {
    List<int> lst(6);
    lst.insert(10);
    lst.insert(20);
    lst.insert(10);
    lst.insert(30);
    lst.insert(20);
    lst.removeDuplicates();
    EXPECT_EQ(lst.search(10), 0);
    EXPECT_EQ(lst.search(20), 1);
    EXPECT_EQ(lst.search(30), 2);
}

TEST(ExtendedListTest_Int, RemoveDuplicatesEmpty) {
    List<int> lst(5);
```

```
    EXPECT_THROW(lst.removeDuplicates(), std::underflow_error);
}

// SubList
TEST(ExtendedListTest_Int, SubListValid) {
    List<int> lst(6);
    lst.insert(5);
    lst.insert(10);
    lst.insert(15);
    lst.insert(20);
    auto sub = lst.subList(1, 2);
    EXPECT_EQ(sub.search(10), 0);
    EXPECT_EQ(sub.search(15), 1);
}

TEST(ExtendedListTest_Int, SubListInvalidCases) {
    List<int> lst(5);
    lst.insert(1);
    lst.insert(2);
    lst.insert(3);
    EXPECT_THROW(lst.subList(5, 6), std::out_of_range); // invalid indices
    EXPECT_THROW(lst.subList(2, 1), std::out_of_range); // start > end
}

TEST(ExtendedListTest_Int, SubListEmpty) {
    List<int> lst(5);
    EXPECT_THROW(lst.subList(0, 1), std::underflow_error);
}

// Find Max/Min
TEST(ExtendedListTest_Int, FindMaxMin) {
    List<int> lst(4);
    lst.insert(25);
    lst.insert(10);
    lst.insert(35);
    EXPECT_EQ(lst.findMax(), 35);
    EXPECT_EQ(lst.findMin(), 10);
}

TEST(ExtendedListTest_Int, FindMaxMinEmpty) {
    List<int> lst(5);
    EXPECT_THROW(lst.findMax(), std::underflow_error);
    EXPECT_THROW(lst.findMin(), std::underflow_error);
}
```

```
// Replace
TEST(ExtendedListTest_Int, ReplaceValid) {
    List<int> lst(5);
    lst.insert(5);
    lst.insert(10);
    lst.insert(5);
    lst.replace(5, 50);
    EXPECT_EQ(lst.search(50), 0);
    EXPECT_EQ(lst.search(10), 1);
}

TEST(ExtendedListTest_Int, ReplaceNotFound) {
    List<int> lst(5);
    lst.insert(1);
    lst.insert(2);
    EXPECT_THROW(lst.replace(3, 10), std::invalid_argument);
}

// ---------------- DOUBLE TESTS ----------------
TEST(ExtendedListTest_Double, SortAndMaxMin) {
    List<double> lst(5);
    lst.insert(2.5);
    lst.insert(1.1);
    lst.insert(3.3);
    lst.sort();
    EXPECT_EQ(lst.findMin(), 1.1);
    EXPECT_EQ(lst.findMax(), 3.3);
}

// ---------------- STRING TESTS ----------------
TEST(ExtendedListTest_String, RotateAndReplace) {
    List<std::string> lst(5);
    lst.insert("apple");
    lst.insert("banana");
    lst.insert("cherry");
    lst.rotateLeft(1);
    EXPECT_EQ(lst.search("banana"), 0);

    lst.replace("banana", "blueberry");
    EXPECT_EQ(lst.search("blueberry"), 0);
}

TEST(ExtendedListTest_String, RemoveDuplicates) {
    List<std::string> lst(6);
    lst.insert("a");
```

```
    lst.insert("b");
    lst.insert("a");
    lst.insert("c");
    lst.removeDuplicates();
    EXPECT_EQ(lst.search("a"), 0);
    EXPECT_EQ(lst.search("b"), 1);
    EXPECT_EQ(lst.search("c"), 2);
}
```